Project BDA: Spring Boot Book Management Application

1. Approach Overview

The goal of this project was to build a robust Spring Boot MVC application to manage two related entities. I selected `Author` and `Book` as the entities to demonstrate a One-to-Many relationship. The application utilizes Spring Data JPA for database interactions, an H2 in-memory database for seamless setup, and JSP with JSTL for the presentation layer.

2. Entity Relationship Design

The project implements a **One-to-Many** relationship between `Author` and `Book`.

- **Author Entity**: Contains `id`, `name`, and `biography`. It has a `@OneToMany` relationship with `Book`, indicating that one author can write multiple books.
- **Book Entity**: Contains `id`, `title`, and `publicationYear`. It has a `@ManyToOne` relationship with `Author`, linking a book to its respective author.

A foreign key `author_id` is created in the `book` table to manage this association.

3. Implementation Details

3.1 Populate Database

An initial data population was implemented using Spring Boot's `CommandLineRunner`. During application startup, it checks if the `Author` table is empty. If so, it creates and saves 10 `Author` records and 10 corresponding `Book` records to the H2 database to ensure the application has sample data out of the box.

3.2 Create Operation

- **View**: A JSP form (`book-form.jsp`) is used to collect the book's title, publication year, and select an existing author from a dropdown.
- **Controller**: The `BookController` handles the POST request to `/books`. It uses `@Valid` to enforce constraints (e.g., non-blank title, valid year).
- **Exception Handling**: Integrity violations (like database constraints) during the save operation are caught, and a user-friendly error message is displayed on the form.

3.3 Read Operation

- **View**: The `list-books.jsp` page displays all books in a styled HTML table using the JSTL `<c:forEach>` tag.
- **Controller**: The `listBooks` method fetches data and binds it to the model.
- **Custom Query**: The `BookRepository` includes a custom query method: `@Query("SELECT b FROM Book b JOIN FETCH b.author")` This performs an inner join fetch to retrieve books and their associated authors in a single query, preventing the N+1 select problem.

3.4 Update Operation

- **View**: The same `book-form.jsp` is re-used for updating. The form fields are pre-populated with the selected book's data.

- **Controller**: A GET method to `/books/edit/{id}` loads the book details. A POST method to `/books/update/{id}` saves the updated information back to the database, utilizing the same exception handling mechanism.

3.5 Testing

Unit tests were written using **JUnit 5** and **Mockito**.

- `BookServiceTest` : Mocks the `BookRepository` to verify the business logic (e.g., verifying `save` and `findAll` behavior).
- `BookRepositoryTest` : Uses `@DataJpaTest` to verify that the custom query method `findAllWithAuthor` correctly executes the inner join and retrieves the data.

4. Challenges Faced and Solutions

- **Challenge**: Resolving the N+1 select problem when displaying a list of books and their authors on the JSP page.
- **Solution**: Implemented the `JOIN FETCH` custom query in the repository layer, which eagerly loads the associated Author for each Book during the initial query.
- **Challenge**: Modern Spring Boot 3.x utilizes Jakarta EE instead of standard Java EE, which breaks traditional `javax.servlet` JSTL dependencies.
- **Solution**: Updated the `pom.xml` to include `jakarta.servlet.jsp.jstl-api` and `tomcat-embed-jasper` to ensure complete compatibility with Spring Boot 3+ embedded Tomcat.

5. Github URL

The source code has been pushed to the required repository: **URL:** https://github.com/Akhil1780/project-bda